Unit 1

Presented by, prof.Kundan Makwana

Data Management concepts

What is Data?

- Data is the basic fact or entity that is utilized in calculation or manipulation.
- There are two different types of data Numeric data and Alphanumeric data.
- When a programmer collects such type of data for processing, he would require to store them in computer’s main memory.
- The process of storing data items in computer’s main memory is called representation.
- Data to be processed must be organized in a particular fashion, these organization leads to structuring of data, and hence the mission to study the Data Structures.

- What is Data Structure?
- Data Structure is a representation of the logical relationship existing between individual elements of data.
- In other words, a data structure is a way of organizing all data items that considers not only the elements stored but also their relationship to each other.
- We can also define data structure as a mathematical or logical model of a particular organization of data items.
- Data Structure mainly specifies the following four things
  - Organization of Data
  - Accessing Methods
  - Degree of Associativity
  - Processing alternatives for information

- The representation of a particular data structure in the memory of a computer is called Storage Structure.
- The storage structure representation in auxiliary memory is called as File Structure.
-

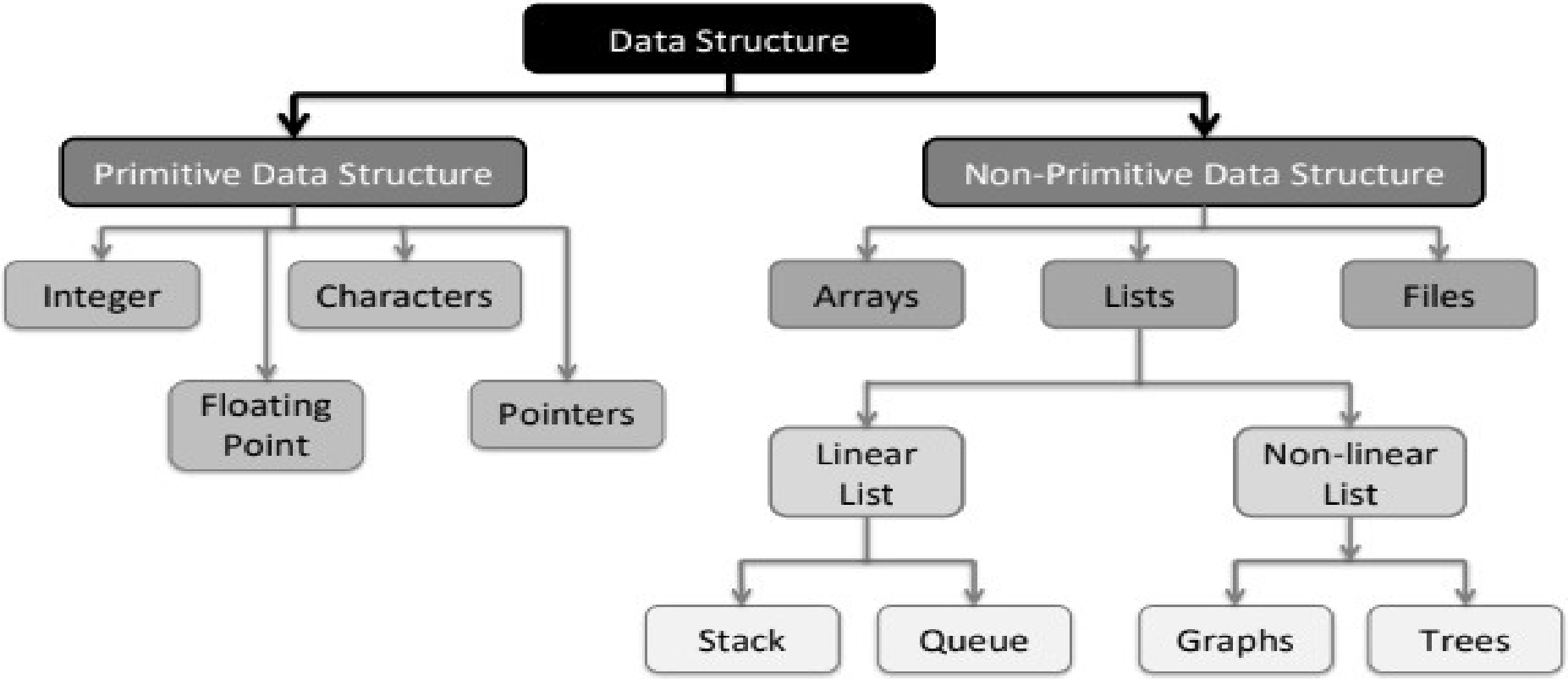
Importance and Advantages of Data Structures

- 1. Efficient Data Organization:
  - Data structures help arrange data in a logical manner, making it easy to access and modify.
  - Example: Using an array to store marks of students instead of separate variables.
- 2. Improves Program Performance
  - Proper data structures reduce time complexity and space usage, leading to faster programs.
  - Example: Binary Search works efficiently only when data is stored in a sorted structure.
- 3. Handles Large Data Easily
  - Modern applications deal with huge data (databases, social media, banking). Data structures make this possible.
  - Example: Trees and graphs are used in databases and networks.
- 4. Supports Reusability and Maintainability
  - Well-defined data structures make programs easier to understand, debug, and modify.
- 5. Foundation of Algorithms
  - Efficient algorithms depend on correct data structures.

Advantages of Data Structures

- 1. Faster Data Access:
    - Operations like searching, insertion, and deletion become faster.
    - Example: Hashing provides almost constant time search.
- 2. Better Memory Management:
    - Data structures use memory efficiently by allocating only what is required.
    - Example: Linked List uses dynamic memory allocation.
- 3. Easy Data Processing:
    - Sorting, searching, and filtering data becomes simple and systematic.
- 4. Improves Data Security:
    - Structured data reduces chances of data corruption and loss.
- 5. Real-World Problem Solving:
    - Data structures model real-life problems easily.
    - Example: Stack → Undo/Redo operation
      Queue → Ticket booking system
      Graph → Google Maps navigation

Classification of Data Structure

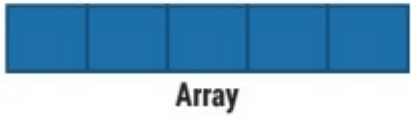


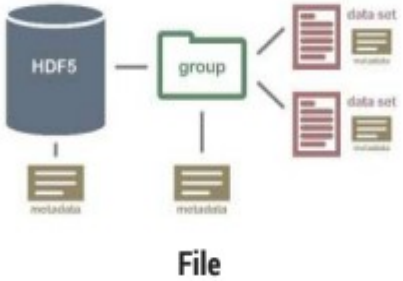
Primitive / Non-primitive data structures

- **Primitive data structures:**
  - Primitive data structures are basic structures and are directly operated upon by machine instructions.
  - **Integers, floats, character and pointers** are examples of primitive data structures.
- **Non primitive data structure:**
  - These are derived from primitive data structures.
  - The non-primitive data structures emphasize on structuring of a group of homogeneous or heterogeneous data items.
  - Examples of Non-primitive data type are Array, List, and File.

Non primitive Data Structure

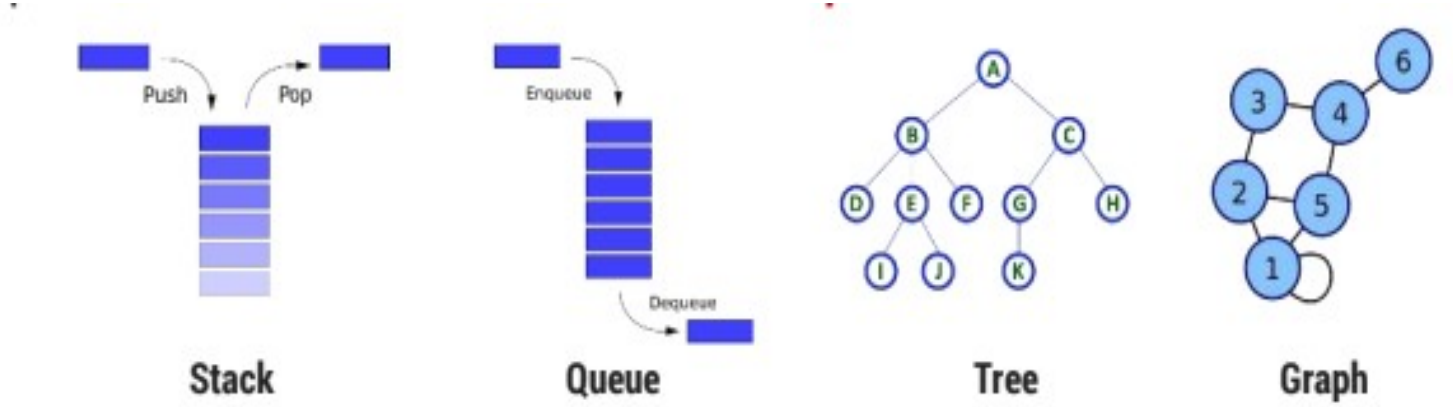
- Array: An array is a fixed-size sequenced collection of elements of the same data type.
- List: An ordered set containing variable number of elements is called as Lists.
- File: A file is a collection of logically related information. It can be viewed as a large list of records consisting of various fields.





Linear / Non-Linear data structure

- **Linear data structures:**
  - A data structure is said to be Linear, if its elements are connected in linear fashion by means of logically or in sequence memory locations.
  - Examples of Linear Data Structure are Stack and Queue.
- **Nonlinear data structures:**
  - Nonlinear data structures are those data structure in which data items are not arranged in a sequence.
  - Examples of Non-linear Data Structure are Tree and Graph.



Operations of Data Structure

- Create: It results in reserving memory for program elements.
- Destroy: It destroys memory space allocated for specified data structure.
- Selection: It deals with accessing a particular data within a data structure.
- Updation: It updates or modifies the data in the data structure.
- Searching: It finds the presence of desired data item in the list of data items.
- Sorting: It is a process of arranging all data items in a data structure in a particular order.
- Merging: It is a process of combining the data items of two different sorted list into a single sorted list.
- Splitting: It is a process of partitioning single list to multiple list.
- Traversal: It is a process of visiting each and every node of a list in systematic manner.

Time and space analysis of algorithms

- Sometimes, there are more than one way to solve a problem.
- We need to learn how to compare the performance of different algorithms and choose the best one to solve a particular problem.
- While analyzing an algorithm, we mostly consider time complexity and space complexity.
- Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input.
- Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.
- Time & space complexity depends on lots of things like hardware, operating system, processors,etc.
- However, we don't consider any of these factors while analyzing the algorithm. We will only consider the execution time of an algorithm.

Asymptotic Notations

- Used to represent time and space complexity.

Notation	Meaning
Big-O (O)	Upper bound (Worst case)
Omega (Ω)	Lower bound (Best case)
Theta (Θ)	Tight bound (Average case)

1. Time Complexity Analysis

- Time complexity measures the amount of time an algorithm takes to run as a function of the input size n.
- Types of Time Analysis
  - **1. Best Case Analysis**
    - Minimum time required for an algorithm.
    - Occurs under the most favorable input.
    - Example:
      - Linear Search
      - Best case: Element found at the first position
      - Time Complexity: $\Omega(1)$

- **2. Average Case Analysis:**
  - Expected time taken over all possible inputs.
  - More realistic than best case.
  - Requires probability distribution of inputs.

  - Example:
    - Linear Search
    - Element found in the middle on average
    - Time Complexity: $\Theta(n)$

- **3. Worst Case Analysis:**
  - Maximum time required for an algorithm.
  - Occurs under the least favorable input.
  - Most commonly used for comparison.
  - Example:
    - Linear Search
    - Element not found or at last position
    - Time Complexity: O(n)

2. Space Complexity Analysis

- **Space complexity measures the amount of memory an algorithm uses.**
- Components of Space Complexity:
  - Fixed Space:
  - Space for constants, variables, and instructions.
- Variable Space:
  - Space for input data.
  - Space for auxiliary data structures.
  - Space for recursive calls (stack space).
- Example:
  - Iterative algorithm → less space
  - Recursive algorithm → more space due to call stack

- **Best Case Space Complexity:**
  - Represents the minimum memory requirement of an algorithm.
  - Occurs when the algorithm executes in the most favorable condition.
  - Uses least auxiliary space.
  - Example:
    - Linear Search
    - If the element is found at the first position, no extra memory is required.
    - Best Case Space Complexity = $\Omega(1)$

- **Worst Case Space Complexity:**
  - Represents the maximum memory required by an algorithm.
  - Occurs in the least favorable condition.
  - Very important because systems must handle this case safely.
  - Example:
    - Recursive function with recursion depth n.
    - Worst Case Space Complexity = O(n)

- **Average Case Space Complexity:**
  - Represents the average memory usage of an algorithm.
  - Calculated by considering all possible inputs.
  - Lies between best and worst case.
  - Example:
    - Recursive function with average recursion depth n/2.
    - Average Case Space Complexity = $\Theta(n)$

- **Average Case Space Complexity:**
  - Represents the average memory usage of an algorithm.
  - Calculated by considering all possible inputs.
  - Lies between best and worst case.
  - Example:
    - Recursive function with average recursion depth n/2.
    - Average Case Space Complexity = $\Theta(n)$

Importance of Performance Analysis

- **Efficient Use of Time and Memory:**
  - Helps in minimizing execution time and memory consumption.
- **Comparison of Algorithms:**
  - Enables selection of the most efficient algorithm among alternatives.
- **Scalability:**
  - Predicts algorithm performance for large input sizes.
- **Improved System Performance:**
  - Leads to faster and more responsive applications.
- **Machine and Language Independent:**
  - Provides a standard method to evaluate algorithms regardless of hardware or language.
- **Reliability in Worst Case:**
  - Ensures the system performs well even under maximum load.
- **Better Design Decisions:**
  - Assists developers in choosing optimal approaches during software design.

Types of Data Structures- Linear & Non Linear Data Structures

- Linear Data Structures:
- Data elements are arranged in a single, sequential path, linked one after another, allowing traversal in a single run.
- Arrays: Contiguous memory, fixed size (often), direct access by index.
- Linked Lists: Nodes linked by pointers, dynamic size, sequential access.
- Stacks: Last-In, First-Out (LIFO) structure (e.g., push, pop).
- Queues: First-In, First-Out (FIFO) structure (e.g., enqueue, dequeue).

- Non-Linear Data Structures:
- Data elements are not sequential; they have hierarchical or interconnected relationships, often at different levels.
- Trees: Hierarchical structure with a root and branches (e.g., Binary Trees, Heaps), good for file systems.
- Graphs: Nodes (vertices) connected by edges, representing complex networks (e.g., social networks, maps).

Difference between Linear and Non-linear Data Structures

S.NO	Linear Data Structure	Non-linear Data Structure
1.	In a linear data structure, data elements are arranged in a linear order where each and every element is attached to its previous and next adjacent.	In a non-linear data structure, data elements are attached in hierarchically manner.
2.	In linear data structure, single level is involved.	Whereas in non-linear data structure, multiple levels are involved.
3.	Its implementation is easy in comparison to non-linear data structure.	While its implementation is complex in comparison to linear data structure.
4.	In linear data structure, data elements can be traversed in a single run only.	While in non-linear data structure, data elements can't be traversed in a single run only.
5.	In a linear data structure, memory is not utilized in an efficient way.	While in a non-linear data structure, memory is utilized in an efficient way.
6.	Its examples are: array, stack, queue, linked list, etc.	While its examples are: trees and graphs.
7.	Applications of linear data structures are mainly in application software development.	Applications of non-linear data structures are in Artificial Intelligence and image processing.
8.	Linear data structures are useful for simple data storage and manipulation.	Non-linear data structures are useful for representing complex relationships and data hierarchies, such as in social networks, file systems, or computer networks.
9.	Performance is usually good for simple operations like adding or removing at the ends, but slower for operations like searching or removing elements in the middle.	Performance can vary depending on the structure and the operation, but can be optimized for specific operations.

Assignment / IMP Questions

- SECTION – A (7 Marks Questions)
  - 1. Explain Data Structures and Data Management concepts in detail.
  - Discuss the need and importance of data management.
  - 2. Explain primitive and non-primitive data types with suitable examples.
  - 3. Explain Performance Analysis and Measurement of algorithms.Describe Time and Space Complexity with best, average and worst case analysis.
  - 4. Explain different types of Data Structures. Differentiate between Linear and Non-Linear Data Structures with examples.
  - 5. Define Data Structure. Explain the importance and advantages of Data Structures in computer applications.

- SECTION – B (Short Answer / 3–4 Marks Questions)
  - 1. What is Data Management? Explain its key concepts.
  - 2. Define Time Complexity and Space Complexity.
  - 3. Explain Best Case, Average Case, and Worst Case analysis.
  - 4. Differentiate between Primitive and Non-Primitive Data Types.
  - 5. Explain Linear Data Structures with examples.
  - 6. Explain Non-Linear Data Structures with examples.

SECTION – C(1 Mark Questions – Very Important)

- 1. Define Data Structure.
- 2. What is Data Management?
- 3. What is Time Complexity?
- 4. What is Space Complexity?
- 5. Define Best Case analysis.
- 6. Define Worst Case analysis.
- 7. What is Average Case analysis?
- 8. Give one example of a Linear Data Structure.
- 9. Give one example of a Non-Linear Data Structure.
- 10.Write any one Primitive Data Type.
- 11.Write any one Non-Primitive Data Type.

Thank You